Take-Home Assignment
Part A: Linear Regression with Gradient Descent

SSE Cost Function, Min-Max Scaling, and Mathematical Derivations

October 26, 2026

1 Data Preprocessing: Min-Max Normalization

1.1 Motivation and Mathematical Formulation

To guarantee smooth gradient descent convergence and prevent uneven distribution on the axis, we apply **Min-Max normalization**. This linear transformation maps all values strictly into the interval $[0, 1]$ according to:

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}, \quad y_{\text{norm}} = \frac{y - y_{\min}}{y_{\max} - y_{\min}} \tag{1}$$

Where the extreme bounds measured on the dataset are:

- $x_{\min} = 6.28, \quad x_{\max} = 28.53$
- $y_{\min} = 427.72, \quad y_{\max} = 489.79$

1.2 Sample Transformed Dataset

Table 1 presents an extract of raw feature values alongside their corresponding Min-Max normalized equivalents.

Table 1: Sample of raw and normalized data values (Min-Max)

Raw Temperature x (°C)	**Raw Energy y (MW)**	**MinMax x_{norm}**	**MinMax y_{norm}**
6.28 ($\min x$)	459.45	0.390562	0.609312
28.53 ($\max x$)	452.76	0.339326	0.501531
23.44	427.72 ($\min y$)	0.768989	0.128242
21.53	489.79 ($\max y$)	0.683146	0.346061
22.68	445.55	0.734831	0.285323
$\vdots$	$\vdots$	$\vdots$	$\vdots$
28.53	427.72	1.000000	0.000000
6.28	489.79	0.000000	1.000000

2 Cost Function Selection: Sum of Squared Errors (SSE)

2.1 Formulation and Justification

For linear regression, the prediction hypothesis for sample i is formulated as:

$$h_\theta(x^{(i)}) = \theta_0 + \theta_1 x^{(i)} \tag{2}$$

We select the *Sum of Squared Errors* (SSE) cost function, defined as $J(\theta_0, \theta_1)$:

$$J(\theta_0, \theta_1) = \frac{1}{2} \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2 \tag{3}$$

Because the SSE cost function is a strictly convex quadratic form with respect to parameters θ_0 and θ_1, this guarantees a single global minimum without local minima traps.

2.2 Relevance of SSE with Min-Max Normalization

Unlike Mean Squared Error (MSE), which divides residual sums by n, SSE accumulates raw squared errors across all n data points.

Without scaling, raw residuals (e.g., 50 MW) produce massive squared errors (2500), resulting in gradients of magnitude 10^5 to 10^6. Working with such high numbers is not relevant when we focus on the most significant decimals.

Min-Max normalization bounds all features and targets within $[0, 1]$, strictly constraining individual residuals $|h_\theta(x^{(i)}) - y^{(i)}| \leq 1$. This allows stable updates with standard learning rates like $\alpha = 0.001$.

3 Derivation of Partial Derivatives and Update Rules

To update parameters via gradient descent, we calculate partial derivatives of $J(\theta)$ with respect to θ_0 (bias) and θ_1 (slope).

3.1 Partial Derivative with respect to θ_0 (Bias)

$$\frac{\partial J(\theta_0, \theta_1)}{\partial \theta_0} = \frac{\partial}{\partial \theta_0} \left[\frac{1}{2} \sum_{i=1}^{n} \left(\theta_0 + \theta_1 x^{(i)} - y^{(i)} \right)^2 \right] \tag{4}$$

$$= \frac{1}{2} \sum_{i=1}^{n} 2 \cdot \left(h_\theta(x^{(i)}) - y^{(i)} \right) \cdot \frac{\partial}{\partial \theta_0} \left(\theta_0 + \theta_1 x^{(i)} - y^{(i)} \right) \tag{5}$$

$$= \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)} \right) \cdot (1) \tag{6}$$

Resulting in:

$$\frac{\partial \mathbf{J}(\theta_\mathbf{0}, \theta_\mathbf{1})}{\partial \theta_\mathbf{0}} = \sum_{\mathbf{i=1}}^{\mathbf{n}} \left(\mathbf{h}_\theta(\mathbf{x^{(i)}}) - \mathbf{y^{(i)}} \right) \tag{7}$$

3.2 Partial Derivative with respect to θ_1 (Slope)

Similarly, applying the Chain Rule for θ_1:

$$\frac{\partial J(\theta_0, \theta_1)}{\partial \theta_1} = \frac{\partial}{\partial \theta_1} \left[\frac{1}{2} \sum_{i=1}^{n} \left(\theta_0 + \theta_1 x^{(i)} - y^{(i)} \right)^2 \right] \tag{8}$$

$$= \frac{1}{2} \sum_{i=1}^{n} 2 \cdot \left(h_\theta(x^{(i)}) - y^{(i)} \right) \cdot \frac{\partial}{\partial \theta_1} \left(\theta_0 + \theta_1 x^{(i)} - y^{(i)} \right) \tag{9}$$

$$= \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)} \right) \cdot (x^{(i)}) \tag{10}$$

Resulting in:

$$\frac{\partial \mathbf{J}(\theta_\mathbf{0}, \theta_\mathbf{1})}{\partial \theta_\mathbf{1}} = \sum_{\mathbf{i=1}}^{\mathbf{n}} \left(\mathbf{h}_\theta(\mathbf{x^{(i)}}) - \mathbf{y^{(i)}} \right) \mathbf{x^{(i)}} \tag{11}$$

3.3 Gradient Descent Update Rules

At each iteration, parameters update simultaneously given learning rate $\alpha > 0$:

$$\theta_0 := \theta_0 - \alpha \frac{\partial J}{\partial \theta_0} = \theta_\mathbf{0} - \alpha \sum_{\mathbf{i=1}}^{\mathbf{n}} \left(\mathbf{h}_\theta(\mathbf{x^{(i)}}) - \mathbf{y^{(i)}} \right) \tag{12}$$

$$\theta_1 := \theta_1 - \alpha \frac{\partial J}{\partial \theta_1} = \theta_\mathbf{1} - \alpha \sum_{\mathbf{i=1}}^{\mathbf{n}} \left(\mathbf{h}_\theta(\mathbf{x^{(i)}}) - \mathbf{y^{(i)}} \right) \mathbf{x^{(i)}} \tag{13}$$

4 Exact Correspondence with Python Implementation

Below is the Python code using `NumPy`, matching the mathematical derivations above.

```python
def hypothesis(X, theta):
    """Returns the hypothetic value"""
    return X @ theta

def compute_derived_cost_theta0(X, Y, theta):
    """Derivation of the SSE cost function relative to the theta0 variable"""
    n = len(Y)
    return np.sum(hypothesis(X, theta) - Y)

def compute_derived_cost_theta1(X, Y, theta):
    """Derivation of the SSE cost function relative to the theta1 variable"""
    n = len(Y)
    return np.sum((hypothesis(X, theta) - Y) * X[:, 1])

def gradient_descent(X, Y, init_theta, iterations, alpha=0.001):
    theta = np.array(init_theta)
    for i in range(iterations):
        d0 = theta[0] - alpha * compute_derived_cost_theta0(X, Y, theta)
        d1 = theta[1] - alpha * compute_derived_cost_theta1(X, Y, theta)

        theta = np.array((d0, d1))

    return theta
```

Listing 1: Implementation of partial derivatives and gradient descent

4.1 Line-by-Line Mathematical Mapping

- **Residual Vector Computation:** The expression `hypothesis(X, theta) - Y` computes the $n \times 1$ vector of individual errors $(h_\theta(x^{(i)}) - y^{(i)})$.
- **Bias Derivative (`compute_derived_cost_theta0`):** The function `np.sum(...)` executes the summation of all n residual values:

$$\texttt{np.sum(hypothesis(X, theta) - Y)} \iff \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)} \right)$$

- **Slope Derivative (`compute_derived_cost_theta1`):** The slicing `X[:, 1]` selects input feature $x^{(i)}$. Element-wise multiplication `*` weights residuals by feature values, followed by `np.sum(...)`:

$$\texttt{np.sum((hypothesis(X, theta) - Y) * X[:, 1])} \iff \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)} \right) x^{(i)}$$

- **Simultaneous Parameter Updates (`gradient_descent`):** Variables `d0` and `d1` compute new parameter states at step $t + 1$ simultaneously before updating array `theta`, adhering strictly to gradient descent rules.